

Docker

Docker este o platformă open-source care permite crearea, rularea și gestionarea containerelor, care sunt unități izolate de aplicații și mediile lor de execuție.

Container

Un **container** este o unitate de lucru izolată, care conține o aplicație și toate bibliotecile și dependențele necesare pentru a rula aplicația în mod consistent, indiferent de mediul de execuție. Containerizarea asigură că aplicația rulează în mod identic pe orice server, reducând problemele de compatibilitate.

Image (Image)

O **image Docker** este un pachet care conține toate fișierele și setările necesare pentru a construi un container. Imaginile sunt citite și nepermit modificări, astfel încât ele reprezintă starea de început a unui container. Poți descărca imagini de la Docker Hub, o platformă publică pentru distribuția de imagini Docker, sau poți crea propriile imagini.

Dockerfile

Un **Dockerfile** este un fișier text cu instrucțiuni pas-cu-pas care permite crearea unei imagini personalizate. Fiecare linie a Dockerfile-ului reprezintă o comandă ce va fi executată pentru a construi imaginea.

Volume

Volumele sunt unități de stocare persistente care permit salvarea datelor create de un container chiar și după ce containerul este șters. Ele sunt utile pentru păstrarea datelor importante.

Docker Compose

Docker Compose este un instrument care permite definirea și gestionarea a mai multor containere în format YAML. Este util pentru orchestrarea aplicațiilor multi-container.

<https://docker-curriculum.com/>

<https://docs.docker.com/get-started/docker-overview/>

<https://www.geeksforgeeks.org/docker-tutorial/>

<https://docs.docker.com/get-started/workshop/>

Build and run an image as a container

Share images using Docker Hub

Deploy Docker applications using multiple containers

<https://docs.docker.com/engine/install/ubuntu/>

```

stud5@node02:~$ docker run -it ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
ff65ddf9395b: Pull complete
Digest: sha256:99c35190e22d294cdace2783ac55effc69d32896daaa265f0bbbedbcde4f3e3e5
Status: Downloaded newer image for ubuntu:latest
root@5be2812a4085:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr
var
root@5be2812a4085:/# ls home
ubuntu
root@5be2812a4085:/# exit
exit
stud5@node02:~$

```

```

stud5@node02:~/tmp6$ docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        latest    59ab366372d5   3 weeks ago    78.1MB
hello-world    latest    d2c94e258dcb   18 months ago  13.3kB
stud5@node02:~/tmp6$

```

```

stud5@node02:~/tmp6$ mkdir java-hello-world
stud5@node02:~/tmp6$ cd java-hello-world/
stud5@node02:~/tmp6/java-hello-world$ nano HelloWorld.java
stud5@node02:~/tmp6/java-hello-world$ ls
HelloWorld.java
stud5@node02:~/tmp6/java-hello-world$ cat HelloWorld.java

```

```

public class HelloWorld {

    public static void main(String[] args) {

        System.out.println("Hello, World from Docker!");

    }

}

```

```

stud5@node02:~/tmp6/java-hello-world$ nano Dockerfile
stud5@node02:~/tmp6/java-hello-world$ cat Dockerfile
# Folosește imaginea oficială OpenJDK ca bază
FROM openjdk:latest

# Creează un director de lucru în container
WORKDIR /app

# Copiază fișierul HelloWorld.java în container
COPY HelloWorld.java /app

# Compilează programul Java
RUN javac HelloWorld.java

# Setează comanda implicită pentru a rula programul
CMD ["java", "HelloWorld"]
stud5@node02:~/tmp6/java-hello-world$ docker build -t java-hello-world .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

```

```

Sending build context to Docker daemon  3.072kB
Step 1/5 : FROM openjdk:latest
latest: Pulling from library/openjdk
197c1adcd755: Pull complete
57b698b7af4b: Pull complete
95a27dbe0150: Pull complete
Digest: sha256:9b448de897d211c9e0ec635a485650aed6e28d4eca1efbc34940560a480b3f1f
Status: Downloaded newer image for openjdk:latest
--> 71260f256d19
Step 2/5 : WORKDIR /app
--> Running in 1beb005e0cbe
Removing intermediate container 1beb005e0cbe
--> 7ae7a7aa9e09
Step 3/5 : COPY HelloWorld.java /app
--> 792ae6aaceac
Step 4/5 : RUN javac HelloWorld.java

```

```
---> Running in 2661d158681b
Removing intermediate container 2661d158681b
---> 187b0d6f192c
Step 5/5 : CMD ["java", "HelloWorld"]
---> Running in 8ef25552bc41
Removing intermediate container 8ef25552bc41
---> 2f5d0b57417a
Successfully built 2f5d0b57417a
Successfully tagged java-hello-world:latest
stud5@node02:~/tmp6/java-hello-world$ docker run java-hello-world
Hello, World from Docker!
stud5@node02:~/tmp6/java-hello-world$
```

Simple Java Client - Server

```
ccostea@node02:~/06/client-server-docker/server$ nano Server.java
ccostea@node02:~/06/client-server-docker/server$ cat Server.java
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(5000)) {
            System.out.println("Server started and listening on port 5000");

            while (true) {
                Socket clientSocket = serverSocket.accept();

                String clientIp = clientSocket.getInetAddress().getHostAddress();
                int clientPort = clientSocket.getPort();
                System.out.println("Client connected: IP = " + clientIp + ", Port = " + clientPort);

                PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
                out.println("Hello from Server!");
                clientSocket.close();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

ccostea@node02:~/06/client-server-docker/server$
ccostea@node02:~/06/client-server-docker/server$ javac Server.java

ccostea@node02:~/06/client-server-docker/server$ java Server
Server started and listening on port 5000
Client connected: IP = 127.0.0.1, Port = 47338
```

```
-----

ccostea@node02:~/06/client-server-docker$ cd client
ccostea@node02:~/06/client-server-docker/client$ nano Client.java
ccostea@node02:~/06/client-server-docker/client$ cat Client.java
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        try (Socket socket = new Socket("127.0.0.1", 5000)) {
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            String message = in.readLine();
            System.out.println("Received from server: " + message);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

ccostea@node02:~/06/client-server-docker/client$ javac Client.java
ccostea@node02:~/06/client-server-docker/client$ java Client
Received from server: Hello from Server!
ccostea@node02:~/06/client-server-docker/client$

-----
```

`docker network` este o comandă care permite crearea, gestionarea și configurarea rețelelor Docker, permite containerelor să comunice între ele, fie că sunt în aceeași rețea, fie că sunt accesate din afara acesteia.

Tipuri de rețele

1. **bridge** - Rețeaua implicită pentru containere independente. Permite containerelor să comunice între ele pe aceeași rețea și este izolată de rețeaua principală a gazdei.
2. **host** - Folosește interfața de rețea a gazdei (host) în loc să izoleze containerele într-o rețea proprie. Containerelor care rulează în rețeaua host vor împărtăși interfața de rețea cu gazda, ceea ce poate fi util pentru aplicații care necesită acces direct la rețeaua gazdei.
3. **none** - Creează un container complet izolat, fără conexiune la nicio rețea. Acest lucru este util pentru containere care nu au nevoie de comunicație de rețea.

```
docker network ls
```

```
docker network create my-network
```

```
docker network connect my-network container_name
```

```
docker network disconnect my-network container_name
```

```
docker network rm my-network
```

```
docker network inspect my-network
```

În Docker Compose, rețelele sunt definite la nivelul serviciilor. Fiecare serviciu poate fi conectat la una sau mai multe rețele. De exemplu, următoarea configurație `.yml` definește o rețea `my-network` pentru două servicii, `client` și `server`, astfel încât acestea să poată comunica între ele:

```
version: '3'
services:
  server:
    build: ./server
    networks:
      - my-network

  client:
    build: ./client
    networks:
      - my-network

networks:
  my-network:
    driver: bridge
```

Docker Network: one image (client + server)

```
ccostea@node02:~/06/client-server-docker/client$ nano Client.java
ccostea@node02:~/06/client-server-docker/client$ cat Client.java
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        try (Socket socket = new Socket("server", 5000)) {
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            String message = in.readLine();
            System.out.println("Received from server: " + message);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
ccostea@node02:~/06/client-server-docker/client$ cd ..
```

```
ccostea@node02:~/06/client-server-docker$ nano server/Dockerfile
```

```
ccostea@node02:~/06/client-server-docker$ cat server/Dockerfile
# Folosește imaginea oficială OpenJDK ca bază
FROM openjdk:latest
```

```
# Creează un director de lucru în container
WORKDIR /app
```

```
# Copiază fișierul Server.java în container
COPY Server.java /app
```

```
# Compilează programul Java
RUN javac Server.java
```

```
# Expune portul 5000 pentru conexiunile clientului
EXPOSE 5000
```

```
# Setează comanda implicită pentru a porni serverul
CMD ["java", "Server"]
```

```
ccostea@node02:~/06/client-server-docker$
```

```
ccostea@node02:~/06/client-server-docker$ nano client/Dockerfile
```

```
ccostea@node02:~/06/client-server-docker$ cat client/Dockerfile
# Folosește imaginea oficială OpenJDK ca bază
FROM openjdk:latest
```

```
# Creează un director de lucru în container
WORKDIR /app
```

```
# Copiază fișierul Client.java în container
COPY Client.java /app
```

```
# Compilează programul Java
RUN javac Client.java
```

```
# Setează comanda implicită pentru a porni clientul
CMD ["java", "Client"]
```

```
ccostea@node02:~/06/client-server-docker$ ls
client  server
```

```
ccostea@node02:~/06/client-server-docker$ nano docker-compose.yml
```

```
ccostea@node02:~/06/client-server-docker$ cat docker-compose.yml
```

```
version: '3'
services:
  server:
    build: ./server
    container_name: java_server
    ports:
      - "5000:5000"
    networks:
      - my-network

  client:
    build: ./client
    container_name: java_client
    depends_on:
      - server
    networks:
      - my-network
```

```
networks:
  my-network:
    driver: bridge
```

```
ccostea@node02:~/06/client-server-docker$
```

```
ccostea@node02:~/06/client-server-docker$ docker-compose up --build
Creating network "client-server-docker_my-network" with driver "bridge"
Building server
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
```

```
Sending build context to Docker daemon 6.144kB
```

```
Step 1/6 : FROM openjdk:latest
```

```
----> 71260f256d19
```

```
Step 2/6 : WORKDIR /app
```

```
----> Using cache
```

```
----> 7ae7a7aa9e09
```

```
Step 3/6 : COPY Server.java /app
```

```
----> 399aa183b75c
```

```
Step 4/6 : RUN javac Server.java
```

```
----> Running in 903e0a145778
```

```
Removing intermediate container 903e0a145778
```

```
----> 431412de4d47
```

```
Step 5/6 : EXPOSE 5000
```

```
----> Running in f3de11095686
```

```
Removing intermediate container f3de11095686
```

```
----> c5d62bcbdcf5
```

```
Step 6/6 : CMD ["java", "Server"]
```

```
----> Running in 88515c4aaa1c
```

```
Removing intermediate container 88515c4aaa1c
```

```
----> 478950458e17
```

```
Successfully built 478950458e17
```

```
Successfully tagged client-server-docker_server:latest
```

```
Building client
```

```
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
```

```
Sending build context to Docker daemon 6.144kB
```

```
Step 1/7 : FROM openjdk:latest
```

```
----> 71260f256d19
```

```
Step 2/7 : WORKDIR /app
```

```
----> Using cache
```

```
----> 7ae7a7aa9e09
```

```
Step 3/7 : COPY Client.java /app
```

```
----> 31489f79c878
```

```
Step 4/7 : COPY wait-for-server.sh /app
```

```
----> 8e07e1bca5cb
```

```
Step 5/7 : RUN chmod +x /app/wait-for-server.sh
```

```
----> Running in b046088464b6
```

```
Removing intermediate container b046088464b6
```

```
----> b659f1181690
```

```
Step 6/7 : RUN javac Client.java
```

```
----> Running in 2b5c8c9e18ad
```

```
Removing intermediate container 2b5c8c9e18ad
```

```
----> 168593c99067
```

```
Step 7/7 : CMD ["/app/wait-for-server.sh"]
```

```
----> Running in 7469bf74b5be
```

```
Removing intermediate container 7469bf74b5be
```

```
----> e6f8d42eb45f
```

```
Successfully built e6f8d42eb45f
```

```

Successfully tagged client-server-docker_client:latest
Creating java_server ... done
Creating java_client ... done
Attaching to java_server, java_client
java_client | Waiting for the server to start...
java_server | Server started and listening on port 5000
java_server | Client connected: IP = 172.22.0.3, Port = 59932
java_client | Received from server: Hello from Server!
java_client exited with code 0
^CGracefully stopping... (press Ctrl+C again to force)
Stopping java_server ... done
ccostea@node02:~/06/client-server-docker$ docker-compose down
Removing java_client ... done
Removing java_server ... done
Removing network client-server-docker_my-network
ccostea@node02:~/06/client-server-docker$ docker image ls
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
client-server-docker_client  latest      e6f8d42eb45f   2 minutes ago  470MB
client-server-docker_server  latest      478950458e17   2 minutes ago  470MB
hello-java           latest      ecc0d69f9838   36 minutes ago  483MB
gospider             latest      5a23de76accc   55 minutes ago  47.3MB
java-hello-world     latest      2f5d0b57417a   11 hours ago    470MB
ubuntu               latest      59ab366372d5   3 weeks ago     78.1MB
golang               latest      17f4a873f350   4 weeks ago     838MB
hello-world          latest      d2c94e258dc8   18 months ago   13.3kB
openjdk              latest      71260f256d19   21 months ago   470MB
alpine               3.17.1      042a816809aa   22 months ago   7.05MB
openjdk              17          5e28ba2b4cdb   2 years ago     471MB
ccostea@node02:~/06/client-server-docker$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS      PORTS          NAMES
f548269cb5e2   gospider       "./gospider --sitema..." 22 minutes ago Exited (1) 22 minutes ago      inspiring_shannon
e1f78a72a571   gospider       "./gospider --sitema..." 22 minutes ago Exited (1) 22 minutes ago      strange_keller
f0a251ca9465   hello-java     "java Hello"              25 minutes ago Exited (0) 25 minutes ago      inspiring_hamilton
24976c8ea06e   gospider       "./gospider -s http:..." 25 minutes ago Exited (0) 25 minutes ago      mystifying_pare
21f1458fecdb   gospider       "./gospider -s http:..." 27 minutes ago Exited (0) 27 minutes ago      naughty_elion
ccostea@node02:~/06/client-server-docker$

```


Docker Network: image client + image server

```
ccostea@node02:~/06/client-and-server$ docker build -t java-server ./server
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
```

```
Sending build context to Docker daemon 3.584kB
Step 1/6 : FROM openjdk:latest
--> 71260f256d19
Step 2/6 : WORKDIR /app
--> Using cache
--> 7ae7a7aa9e09
Step 3/6 : COPY Server.java /app
--> c7db66de80db
Step 4/6 : RUN javac Server.java
--> Running in f2664c4fd7a2
Removing intermediate container f2664c4fd7a2
--> 344820feb274
Step 5/6 : EXPOSE 5000
--> Running in ef1063dcb37c
Removing intermediate container ef1063dcb37c
--> 14c89b8702b3
Step 6/6 : CMD ["java", "Server"]
--> Running in 093ed5c269b5
Removing intermediate container 093ed5c269b5
--> d59831053111
Successfully built d59831053111
Successfully tagged java-server:latest
```

```
ccostea@node02:~/06/client-and-server$ ls -al
total 16
drwxrwxr-x 4 ccostea ccostea 4096 Nov  5 08:37 .
drwxrwxr-x 4 ccostea ccostea 4096 Nov  5 08:36 ..
drwxrwxr-x 2 ccostea ccostea 4096 Nov  5 08:44 client
drwxrwxr-x 2 ccostea ccostea 4096 Nov  5 08:37 server
ccostea@node02:~/06/client-and-server$ ls server
Dockerfile  Server.java
```

```
ccostea@node02:~/06/client-and-server$ cat client/Client.java
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) {
        try (Socket socket = new Socket("java_server", 5000)) {
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            String message = in.readLine();
            System.out.println("Received from server: " + message);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
ccostea@node02:~/06/client-and-server$ docker build -t java-client ./client
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
```

```
Sending build context to Docker daemon 3.072kB
Step 1/5 : FROM openjdk:latest
--> 71260f256d19
Step 2/5 : WORKDIR /app
--> Using cache
--> 7ae7a7aa9e09
Step 3/5 : COPY Client.java /app
--> 94395754e2fc
Step 4/5 : RUN javac Client.java
--> Running in 85a6acf4eea1
Removing intermediate container 85a6acf4eea1
--> 03084e1afff1
Step 5/5 : CMD ["java", "Client"]
```

```
---> Running in 0079c0ea0035
Removing intermediate container 0079c0ea0035
---> 4c00f8254eb1
Successfully built 4c00f8254eb1
Successfully tagged java-client:latest
```

```
ccostea@node02:~/06/client-and-server$ docker run --rm --name java_client --network my-network java-client
Received from server: Hello from Server!
ccostea@node02:~/06/client-and-server$
```

```
ccostea@node02:~/06/client-and-server$ docker run --rm --name java_client --network my-network java-client
Received from server: Hello from Server!
ccostea@node02:~/06/client-and-server$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
java-client	latest	4c00f8254eb1	2 minutes ago	470MB
java-server	latest	d59831053111	11 minutes ago	470MB
gospider	latest	5a23de76accc	2 hours ago	47.3MB
java-hello-world	latest	2f5d0b57417a	11 hours ago	470MB
ubuntu	latest	59ab366372d5	3 weeks ago	78.1MB
golang	latest	17f4a873f350	4 weeks ago	838MB
openjdk	latest	71260f256d19	21 months ago	470MB
alpine	3.17.1	042a816809aa	22 months ago	7.05MB
openjdk	17	5e28ba2b4cdb	2 years ago	471MB

```
ccostea@node02:~/06/client-and-server$
```

```
ccostea@node02:~/06/client-and-server$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
859c3764b679	java-server	"java Server"	4 minutes ago	Up 4 minutes	5000/tcp	java_server

```
ccostea@node02:~/06/client-and-server$
```

